Python Engineering - CSV Round-Trip Testing Guide

What test_csv_round_trip() validates, why changing reference does not cause FAIL, and how to break it correctly

1. What is a round trip?

A **round trip** checks that data can leave the program, pass through an external representation, and return without changing its structure or values.

```
DATA IN RAM
    |
    v
save_trajectory_csv()
    |
    v
CSV ON DISK
    |
    v
load_trajectory_csv()
    |
    v
RECOVERED DATA IN RAM
```

The test asks: **is what I recover equivalent to what I originally saved?**

2. What test_csv_round_trip() does NOT check

This test does not verify whether the trajectory is physically correct. It does not know whether altitude should be 100, 999, or 5000. It only checks that save/load preserves the supplied data.

```
round-trip test checks:
original data ~= loaded data

it does NOT check:
original data == physically correct trajectory
```

Physical correctness belongs to other tests, such as test_euler_step().

3. Test logic

```
reference = np.array([
    [100.0, 20.0, 0.0],
    [102.0, 19.019, 0.1]
])

save_trajectory_csv(test_file, reference)
loaded = load_trajectory_csv(test_file)

assert reference.shape == loaded.shape
assert np.allclose(reference, loaded)
```

Check	Question
shape	Are rows and columns preserved?
np.allclose	Are numerical values preserved?

4. Why does changing 100 to 999 not produce FAIL?

Because reference plays two roles: it is the data being saved and also the data we expect to recover.

```
reference = np.array([
    [999.0, 20.0, 0.0],
    [102.0, 19.019, 0.1]
])
```

```
input reference  -> 999
stored CSV      -> 999
loaded value    -> 999
comparison      -> 999 ~= 999
result          -> PASS
```

The test is behaving correctly: it preserved the value it was given.

5. Reference is not the same as a physical expected value

Concept	Meaning
reference in round trip	Input data that must survive save + load
expected in test_euler_step	Independent physical/numerical result expected from the model

Changing expected in a physics test can cause FAIL because the calculation is checked against an external reference. Changing reference in a round trip changes both input and recovery expectation.

6. Physical test example

```
state = VerticalState(h=100.0, v=20.0)
state.euler_step(0.1)

result = state.state_vector()
expected = np.array([102.0, 19.019, 0.1])

assert np.allclose(result, expected)
```

Here expected comes from the mathematical model. Changing it to 999 without changing the real result should make the test fail.

7. What test_csv_round_trip() is really for

Its purpose is to detect serialization/deserialization errors, such as wrong column order, missing data, formatting problems, or inconsistent parsing.

```
- swap v and t while saving
- omit one column
- duplicate a column
- read columns in the wrong order
- modify values during parsing
- change the number of rows
```

8. Example: swapped columns

```
# wrong writer
file.write(
    f'{h:.6f},{t:.6f},{v:.6f}\n'
)
```

Original:

```
[100.0, 20.0, 0.0]
[102.0, 19.019, 0.1]
```

Incorrect file:

```
100.000000,0.000000,20.000000
102.000000,0.100000,19.019000

np.allclose(reference, loaded)
-> False
-> AssertionError
-> FAIL
```

This is the kind of regression a round-trip test is intended to detect.

9. How to deliberately produce FAIL

To prove that the test detects changes, create a mismatch between the original input and the recovered output.

```
loaded = load_trajectory_csv(test_file)

# deliberate corruption for learning
loaded[0, 0] = 999.0

assert np.allclose(reference, loaded)
```

If reference[0,0] is still 100, the assertion fails and the runner reports FAIL.

10. Another method: temporarily break the writer

A more realistic demonstration is to temporarily introduce a bug in save_trajectory_csv(), for example by swapping columns.

```
correct:
h, v, t

temporary bug:
h, t, v
```

Run the tests, verify that test_csv_round_trip() fails, then restore the correct writer and confirm PASS again.

11. Why deliberate failure is useful

A test that always passes may itself be poorly designed. Deliberately introducing the type of bug it is meant to detect confirms that the test is sensitive to that failure mode.

```
working code -> PASS
deliberate bug -> FAIL
restore code -> PASS
```

This cycle validates both the code and the usefulness of the test.

12. What question does each EX24 test answer?

Test	Question
test_initial_state()	Does VerticalState initialize correctly?
test_euler_step()	Does the Euler step produce the expected result?
test_csv_round_trip()	Is what I save the same as what I load?
test_missing_file_raises()	Does a missing file raise the correct exception?

13. One test = one clear behavior

```
physics test      -> numerical behavior
round-trip test   -> persistence integrity
exception test    -> failure behavior
```

A clear test question makes failures easier to diagnose.

14. Round trip and serialization

Saving a NumPy array to CSV converts numerical values into text. Loading performs the inverse transformation.

```
NumPy float
   |
serialization
   v
CSV text
   |
parsing / deserialization
   v
NumPy float
```

The round trip verifies that both transformations are compatible.

15. Important limitation of round-trip testing

If writer and reader make the same symmetric mistake, a round-trip test may still pass. For example, if both swap the same columns, the final data may appear unchanged.

Real projects therefore combine round-trip tests with known-value tests, raw-format checks and independent validation.

16. When to use an additional known-format test

If you also want to verify that the physical CSV text has an exact format, create a separate test that reads raw text and compares expected lines.

```
save_trajectory_csv(...)
read raw text
compare against expected CSV lines
```

That is a different behavior and should ideally be a different test.

17. Concept summary

```
test_euler_step()
-> compare calculation with known physical result

test_csv_round_trip()
-> compare loaded data with original input

changing reference in round trip
-> changes both saved input and expected recovery
-> therefore it may still PASS

to force FAIL
-> create a mismatch between original and loaded data
```

18. Final cheat sheet

```
ROUND TRIP
original
 -> save
 -> file
 -> load
 -> recovered

CHECK 1
original.shape == recovered.shape

CHECK 2
np.allclose(original, recovered)

PASS
-> structure and values preserved

FAIL
-> something changed during save/load
```

Core idea: a round-trip test does not ask whether the data is physically correct; it asks whether the persistence system preserves it correctly.